

PA3

jerrycyhu

24. März 2024

1 Problem Statement

PA3 involves implementing a Convolutional Neural Network (CNN) for image classification on the CIFAR-10 dataset. CIFAR-10 contains 60,000 32x32 color images of objects across 10 classes, with 50,000 images designated for training and 10,000 for testing. However, for this assignment, we are provided with a subset of 50,000 training images and 5,000 testing images.

The task includes the following key components:

Data Preparation: Preprocess the data by normalizing the image intensities ($training_data = training_data / 255.0$) and converting the labels to a one-hot encoded format.

Model Architecture: The layer includes:

Input Layer: Output is 3x32 x32

Convolutional Layer 1: 16 filters of size 5x5, with a stride of 1 and no padding.

Max Pooling Layer 1: 2x2 pooling area with a stride of 2.

Convolutional Layer 2: 32 filters of size 5x5, with a stride of 1 and no padding.

Max Pooling Layer 2: 2x2 pooling area with a stride of 2.

Convolutional Layer 3: 64 filters of size 3x3, with a stride of 1 and no padding.

Fully Connected Layer: Input is 576, with output as 500.

Output Layer: Input is 500 with output as 10.

Batch Processing: Utilize the DataLoader class to split the training dataset into random batches.

Backpropagation: Employ SGD (Stochastic Gradient Descent) for optimization, with a specified learning rate, and utilize the backward propagation method for computing gradients.

Model Saving

While the past PAs are more theoretical, PA3 emphasizes the practical application of deep learning techniques for image classification.

2 Model Description

2.1 Input Layer

The model accepts images with 3 color channels (RGB) as input. The exact input image size is 32x32. So the total output is 3x32x32 (While it should actually be 32x32x3, the PyTorch accepts the tensor with the channel number at the front, so the actual dimension is 3x32x32)

2.2 Convolutional and Pooling Layers

2.2.1 Conv1

The first convolutional layer has 3 input channels, 16 output channels, a kernel size of 5x5, and a stride of 1. It's followed by a ReLU activation function.

2.2.2 Pool1

A max-pooling layer with a 2x2 kernel and stride of 2 is applied to reduce the spatial dimensions of the feature maps.

After the Conv1 and Pool1, the output dimension will be 16x14x14

2.2.3 Conv2

The second convolutional layer increases the depth to 32 output channels, with a 5x5 kernel and a stride of 1, also followed by ReLU. For a single sample, the output dimension will be 16x14x14 after Conv1.

2.2.4 Pool2

A max-pooling layer with a 2x2 kernel and stride of 2 is applied to reduce the spatial dimensions of the feature maps.

After the Conv2 and Pool2, the output dimension will be 32x5x5

2.2.5 Conv3

The third convolutional layer further increases the depth to 64 output channels, with a 3x3 kernel and a stride of 1, also followed by ReLU.

After the Conv3, the output dimension will be 64x3x3

2.3 Fully Connected Layers:

2.3.1 FC1

After flattening the output from the last convolutional layer, the first fully connected layer has $64 \times 3 \times 3 = 576$ inputs and 500 output features, followed by

ReLU.

2.3.2 FC2

The second fully connected layer maps the 500 features to the number of classes (`num_classes`) in the dataset and applies a softmax activation to produce a probability distribution over the classes.

3 Experimental Setup

3.1 Dataset Preparation and Split

3.1.1 Dataset Loading

I created a custom dataset class (`My_dataset`) for loading and preprocessing your training and testing data. This class handles the conversion of images and labels into PyTorch tensors and formats the images for the model (e.g., permutes image dimensions so that the channels are the first axis.).

3.1.2 Training and Testing Data

I load my training and testing data from `.npy` files (`training_data.npy`, `training_label.npy`, `test_data.npy`, `test_label.npy`).

3.1.3 Data Normalization

I normalize the image data by dividing by 255.0, scaling the pixel values to the range $[0, 1]$ to facilitate model training.

3.2 Model Training

3.2.1 Batch Size

I set the batch size to 64, which is a common choice for training neural networks.

3.2.2 Weight Initialization

I didn't specify custom weight initialization for the model layers. PyTorch's default initialization method for each layer type is used. According to the PyTorch Support Document, convolutional layers and linear layers in PyTorch have their own default initialization methods that are generally suitable for a wide range of tasks.

3.2.3 Learning Rate and Adjustment

The initial learning rate is set to 0.1, and I used Stochastic Gradient Descent (SGD) as my optimizer with a weight decay of 0.004. I implement a dynamic adjustment strategy for the learning rate based on the

change in loss between epochs. If the loss decreases, it will speed up the learning speed even more by increasing the learning rate by a factor of 1.05; if the loss increases, it will decrease the learning rate by a factor of 0.5, learning more carefully with small steps. This approach is intended to adaptively find a better learning rate during training for potentially faster convergence.

The adaptive learning rate approach speeded up the training speed by almost 10 times!

3.2.4 Training Loop

The model trains until the change in loss is less than 0.0001 (and positive) or the accuracy surpasses 80%. This approach ensures that training continues until the model either achieves a satisfactory level of accuracy or stops making significant progress in reducing the loss.

3.3 Testing

3.3.1 Model Evaluation

After training, I evaluate the model on the test dataset to assess its performance. The evaluation includes calculating the average loss and the overall accuracy (i.e., the proportion of correctly classified instances in the test set).

3.4 Custom Functions:

3.4.1 Custom Cross-Entropy Loss

I implemented a custom cross-entropy loss function, adding a small epsilon to the predictions to prevent $\log(0)$ calculations that would result in numerical instability.

3.4.2 Accuracy Calculation

A custom function calculates the model's accuracy by comparing the predicted class (the class with the highest probability) against the true class labels.

4 Experimental results

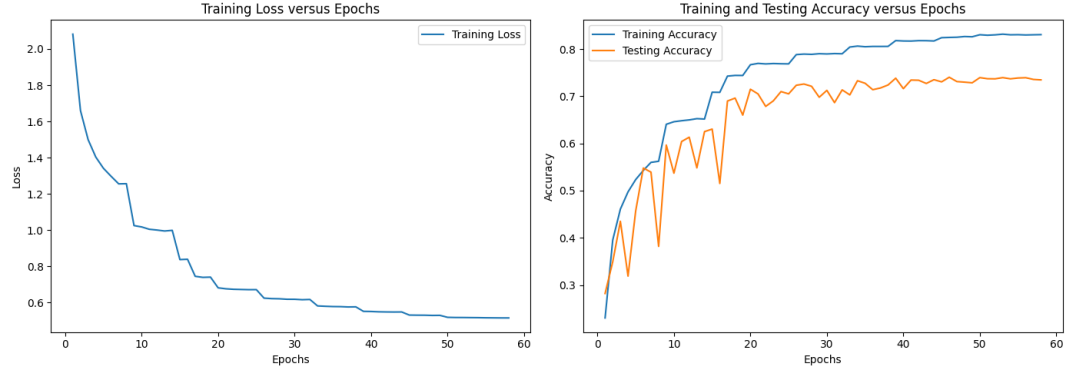


Fig 1. Training Loss versus Epochs(Left), Training and Testing Accuracy versus Epochs(Right)

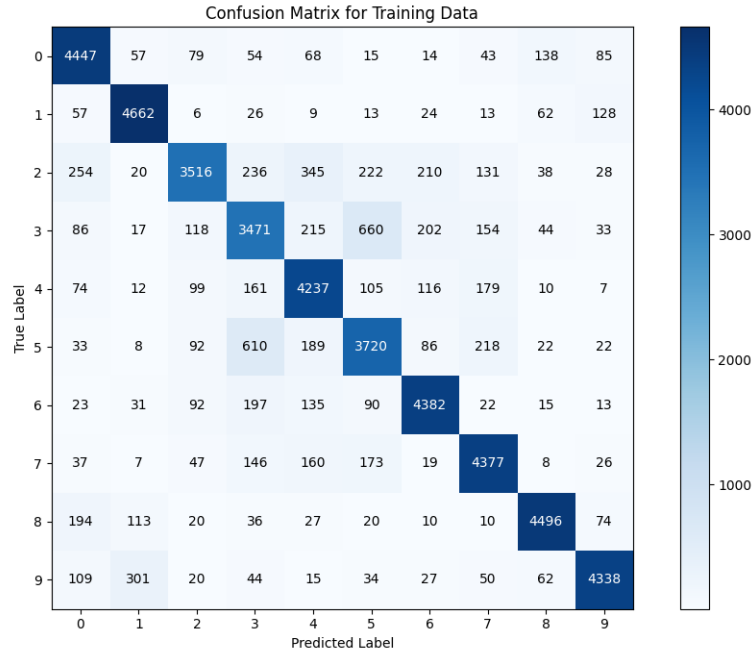


Fig 2. Confusion Matrix for Training Data

4.1 Average Classification Error

Class 0 Error: 0.2152
 Class 1 Error: 0.1327
 Class 2 Error: 0.4570
 Class 3 Error: 0.4346

Class 4 Error: 0.2742
 Class 5 Error: 0.3770
 Class 6 Error: 0.1609
 Class 7 Error: 0.1859
 Class 8 Error: 0.1667
 Class 9 Error: 0.2476
 Average Classification Error: 0.2652

5 Analysis



Fig.3. Filter matrix of Conv1. Note that I normalized the filter matrix to fit in the RGB visualization, otherwise the matrix visualization will be very dark and will lose many details.

An intriguing observation is the intense drift in the testing accuracy as compared to the training accuracy. This can be attributed to the implementation of an adaptive learning rate, which escalates until a rise in loss is detected. Consequently, there are instances during training when the loss may indeed rise. Such an uptick in loss is indicative of a diminishing generalization capability of the model. Although the impact on the training dataset is marginal, this reduction

in generalizability appears to be significantly more pronounced when the model is evaluated against testing data, which it has not previously encountered. This inference is supported by Figure 1 (left), where a marked decrease in testing accuracy aligns with a noticeable deceleration in the rate at which training accuracy improves. While the accuracy for the training data merely shows a deceleration in improvement rate, the testing data presents a substantial drop in accuracy.

6 Conclusion

In this PA, I made a comprehensive evaluation of a machine learning model's performance over numerous training epochs. The model was trained with a dynamic learning rate that adjusted in response to loss trends. The performance metrics were carefully monitored, including the training loss and both the training and testing accuracy.

The findings indicate that the model's learning rate adaptation plays a crucial role in its ability to generalize. Notably, the fluctuating testing accuracy suggests the model may be experiencing overfitting or instability during training, but this is intentionally made since the model is designed to always explore the currently best learning rate (biggest learning step). The visual data presented in Figure 1 (left) was pivotal in understanding these dynamics, revealing a correlation between changes in the learning rate and subsequent model performance on unseen data.